How We Build Our Own Little Google

The super-simple, step-by-step guide for our
Big Data mini-project: **Distributed Search Engine**

P1	P2	P3	P4
The Doorman	The Librarian	The Counting Team	The Help Desk
Node 1	**Node 2**	**Node 3**	**Node 4**
Ingestion	Storage	Compute	Search

Write your names here:

P1 (Node 1)	P2 (Node 2)	P3 (Node 3)	P4 (Node 4)

How to read this guide

Grey boxes are commands: type them **exactly**. The coloured pill (P1 P2 P3 P4 ALL) says **who** types it. Green boxes tell you how to know it worked. Blue boxes are what to say to the teacher. Tick the little boxes as you go.

1. The story: what are we even building?

Imagine a giant library with 1,000 books (our Wikipedia pages). People want to ask a question like "*volcano*" and get the **best** books first, in less than a blink. One person can't do all the work, so we split it between **4 computers** (called **nodes**), and each of us looks after one.

Node 1 · P1
The Doorman checks every new book, throws out torn ones, keeps only the newest edition

→

Node 2 · P2
The Librarian keeps two copies of every book and a photo album of every change

→

Node 3 · P3
The Counting Team makes the "index at the back of the book" and a popularity score

→

Node 4 · P4
The Help Desk you ask a question, it hands you the best books

The big words, in kid language

Big word	What it really means
ZeroTier	A secret walkie-talkie channel so our 4 computers can talk to each other, even if they are in different houses.
HDFS	A shared toy box spread over Node 2 and Node 3. Every toy is kept **twice** (replication = 2), so if one box breaks, nothing is lost.
NameNode / DataNode	The NameNode is the notebook that says *which box has which toy* (Node 2). DataNodes are the boxes themselves (Node 2 and Node 3).
Spark	A group of helpers who share a big job. The **Master** (Node 3) is the teacher handing out work; **Workers** (Node 2 and Node 3) do it. If a helper faints, the teacher gives the work to someone else.
Apache Iceberg	A smart way of keeping tables that takes a **photo (snapshot)** after every change. You can look at old photos (time travel) or go back to one (rollback).
Inverted index	The index at the back of a book: "*volcano → pages 4, 17, 230*". Searching becomes super fast.
PageRank	A popularity score. A page is popular if popular pages link to it. Like being invited to parties by the popular kids.
BM25	A score for "how well does this page match the words you typed?". Saying a word 100 times doesn't help much, and long pages don't get to cheat.
Parquet	A neat, packed file format for tables. Node 3 packs the results into Parquet and Node 4 unpacks them.
FastAPI	The little Python program on Node 4 that answers search questions from the web page.

Final score formula (from the project sheet)
Final = 0.7 × (BM25 ÷ biggest BM25) + 0.3 × (PageRank ÷ biggest PageRank). BM25 uses k1 = 1.2, b = 0.75. PageRank uses damping 0.85 and at least 10 rounds.

2. The 6 golden rules (read these out loud together)

1. **Same username everywhere.** All 4 computers use the same login name (we use `ubuntu`).
2. **Never format the NameNode twice.** Only P2 does it, only once. Doing it again erases the whole toy box.
3. **Nodes 2 and 3 must stay awake** whenever anyone runs a job. Node 2 + Node 3 asleep = nothing works.
4. **Don't change shared files alone.** `cluster.env` and the `conf/` folder are shared. Tell the group first (P2 is the boss of these).
5. **One tokenizer, two copies.** `jobs/textutil.py` and `search/textutil.py` must be twins. After changing one, run `bash scripts/sync_textutil.sh`.
6. **Always run commands from inside the `dse` folder** (type `cd ~/dse` first).

3. Day 1: everybody together ALL

Sit together (or on a call). Every person does these on **their own** computer, at the same time.

1. **Get the project.** Put the `dse` folder on every computer (copy the zip or `git clone` your team repo).
2. **Make the walkie-talkie channel.** P2 goes to `my.zerotier.com`, clicks *Create A Network*, and shares the 16-letter **Network ID** with everyone.
3. **Everyone joins it:**

   ```
   curl -s https://install.zerotier.com | sudo bash
   sudo zerotier-cli join <NETWORK_ID>
   ```

   P2 then ticks **Auth** for all 4 machines on the website and gives them fixed IPs, ending in .11, .12, .13 and .14.
4. **Fill in the address book.** P2 types the 4 real IPs into `cluster.env` and sends that file to everyone. All 4 copies must be identical.
5. **Give each computer its name** (N = your node number: P1 types 1, P2 types 2, and so on):

   ```
   cd ~/dse
   sudo bash scripts/setup_hosts.sh N
   ```

   It should say **reachable** four times.
6. **Install everything** (about 10 minutes; go get a snack):

   ```
   sudo bash scripts/install_node.sh N
   source /etc/profile.d/bigdata.sh
   ```

You did it right if...
`ping node1`, `ping node2`, `ping node3` and `ping node4` all answer, from every computer.

3. Day 1 (continued): wake up the cluster

7. P2 **first** starts the toy box (HDFS) and a Spark helper:

```
bash scripts/start_node.sh 2
```

8. P3 right after starts the Spark teacher (Master), a helper and the second toy box:

```
bash scripts/start_node.sh 3
```

9. P2 checks the whole team is alive:

```
bash scripts/check_cluster.sh
```

10. P2 creates the empty Iceberg tables (only once, ever):

```
bash scripts/submit.sh jobs/init_tables.py
```

11. P1 looks at the real data files and tells everyone the key names:

```
python3 scripts/inspect_dataset.py ~/data/pages.jsonl ~/data/changes.jsonl
```

If a key name isn't in the `FIELD_ALIASES` list at the top of `jobs/ingest.py`, add it there.

You did it right if...

- `check_cluster.sh` says **Live datanodes (2)** and shows **2 ALIVE** workers.
- Typing `jps` on Node 2 shows *NameNode, DataNode, Worker*.
- On Node 3, `jps` shows *Master, Worker, DataNode, HistoryServer*.
- The web pages open: `http://node2:9870` (toy box), `http://node3:8080` (Spark teacher), `http://node3:18080` (Spark history).

Uh-oh: "Live datanodes (0)"?

Almost always the `/etc/hosts` file. Run `setup_hosts.sh` again on that computer, then:

```
hdfs --daemon stop datanode
hdfs --daemon start datanode
```

Uh-oh: Node 2's worker isn't in the list?

It woke up before the teacher. On Node 2 run:

```
stop-worker.sh
start-worker.sh spark://node3:7077
```

Make 4 promises before anyone codes

Promise	Between	What we agree
Input key names	P1 & all	The names `inspect_dataset.py` printed, put in `FIELD_ALIASES`
Table columns	P1 P2 & P3	As written in `jobs/init_tables.py`. Nobody changes them silently.
Export shape	P3 & P4	`/exports/vN/` holds `docs/`, `postings/` and `manifest.json`
Tokenizer	P3 & P4	Only one person edits `textutil.py`, then runs `sync_textutil.sh`

P1 The Doorman: Node 1 (Ingestion)

Your job in one sentence: take the raw data file, throw away the broken lines, keep only the newest version of each page, and put the clean pages into the Iceberg table.

Your files: `jobs/ingest.py` · `scripts/node1_ingest_phase.sh` · `scripts/inspect_dataset.py`. Your computer runs **no** servers; it is only the "boss" (driver) of the ingestion job.

Phase 0: the first big pile of books

1. Put `pages.jsonl` and `changes.jsonl` in `~/data/` on Node 1.
2. Run the doorman:

   ```
   cd ~/dse
   bash scripts/node1_ingest_phase.sh 0 ~/data/pages.jsonl
   ```

   It uploads the file to HDFS (2 copies), then sends the Spark job to the teacher on Node 3.
3. Read the summary at the end: how many lines were read, how many were rejected and why, and how many duplicates were dropped.
4. Tell **P3**: "*Phase 0 is in, go!*"

Phase 1: the changes (only after Phase 0 has been searched on Node 4)

5. ```
   bash scripts/node1_ingest_phase.sh 1 ~/data/changes.jsonl
   ```

   Newer versions replace old ones, deletes remove pages, and old (stale) versions are ignored.
6. **Magic trick (idempotent):** run the exact same command again. It should say `No page changes -> skipping MERGE`. Running it twice changes nothing.
7. Tell **P3**: "*Phase 1 is in, go!*"

You did it right if...

the log ends with `DONE. pages snapshot ... -> ...` and shows the `ingest_log` table. The second Phase 1 run shows only `noop`.

What to say to the teacher

"Every line goes through `json.loads` on the workers. Bad lines go to a **rejects** table with a reason, like *malformed_json* or *missing_url*. For duplicates I keep the **highest version**, using a window function. Then I compare against the current Iceberg table, so only real changes go into one **MERGE INTO**, which is one atomic snapshot. That's why running a phase twice changes nothing. Finally I drop links that point to pages that don't exist."
   ```

P2 The Librarian: Node 2 (Storage + Lakehouse)

Your job in one sentence: keep the shared toy box (HDFS) healthy with 2 copies of everything, and look after the Iceberg tables and their photo album (snapshots).

Your files: `conf/core-site.xml` · `conf/hdfs-site.xml` · `cluster.env` · `jobs/init_tables.py` · `jobs/maintenance.py`. You run: NameNode, DataNode, Spark Worker.

1. **Every morning**, make sure your machine is on and run `bash scripts/check_cluster.sh`. If it was rebooted, run `bash scripts/start_node.sh 2` again. It will **not** re-format; it is safe.

2. **Prove "2 copies":**

   ```
   hdfs fsck /input -files -blocks -locations
   ```

   Look for `repl=2` and both node2 and node3 in the locations.

3. **Show the photo album** after each phase:

   ```
   bash scripts/submit.sh jobs/maintenance.py snapshots
   ```

   Write down the pages snapshot IDs after Phase 0 (call it S0) and after Phase 1 (call it S1).

4. **Time travel:** what changed between the two photos?

   ```
   bash scripts/submit.sh jobs/maintenance.py diff --from S0 --to S1
   ```

5. **What did the doorman throw away?**

   ```
   bash scripts/submit.sh jobs/maintenance.py rejects
   ```

6. **Undo a bad update (rollback)**, for practice:

   ```
   bash scripts/submit.sh jobs/maintenance.py rollback --table pages --snapshot S1
   ```

   Afterwards, P3 must re-run `node3_process.sh` so the search matches.

7. **Toy-box-breaks demo.** Ask P3 to stop Node 3's DataNode (`hdfs --daemon stop datanode`). Then show that reading still works:

   ```
   hdfs dfs -cat /input/pages.jsonl | head -2
   hdfs fsck /
   ```

   You'll see "under-replicated". Then P3 starts it again with `hdfs --daemon start datanode`.

You did it right if...
you can show *Live datanodes (2)*, *repl=2*, a list of snapshots, and a diff between S0 and S1.

What to say to the teacher
"HDFS keeps every block on both DataNodes (replication 2), so losing one DataNode loses no data. Iceberg stores its data *and* metadata in HDFS. Every commit is a snapshot, so we can time travel with `VERSION AS OF` and roll back a bad update. One honest weak spot: the NameNode on Node 2 is a single point of failure."

P3 The Counting Team: Node 3 (Compute)

Your job in one sentence: be the Spark teacher, build the "index at the back of the book", compute PageRank, check everything, and pack the results for Node 4.

Your files: `conf/spark-defaults.conf` · `jobs/index.py` · `jobs/pagerank.py` · `jobs/export.py` · `scripts/node3_process.sh`. You run: Spark Master, Spark Worker, DataNode, History Server.

1. When P1 says "go", run all three jobs in a row (it stops by itself if anything fails):

   ```
   cd ~/dse
   bash scripts/node3_process.sh
   ```

   Or one at a time:

   ```
   bash scripts/submit.sh jobs/index.py
   bash scripts/submit.sh jobs/pagerank.py --iterations 20
   bash scripts/submit.sh jobs/export.py
   ```

2. **Index:** the first time it builds everything. After Phase 1 it only redoes the pages that changed. It finds them by comparing two Iceberg snapshots. Look for `INDEX DONE (incremental)`.

3. **PageRank:** watch the "L1 change" number get smaller each round (it is settling down). At the end, the **sum of ranks** should be about **1.0**.

4. **Export:** it checks everything first. Only if all is good does it publish `/exports/v1` (then v2 after Phase 1). Tell P4 : "*v1 is ready!*"

5. **Helper-faints demo (Spark fault tolerance):** start a long run:

   ```
   bash scripts/submit.sh jobs/pagerank.py --iterations 30
   ```

   While it runs, P2 kills a helper on Node 2:

   ```
   kill -9 $(jps | awk '/CoarseGrainedExecutorBackend/{print $1}')
   ```

   Open `http://node3:4040`: the executor is "lost", its tasks are redone, and the job still finishes.

6. **Where did the time go?** Open `http://node3:18080`, click a job, and show stage times and *Shuffle Read/Write* (the network traffic).

You did it right if...
`hdfs dfs -ls /exports` shows `v1` (then `v2`), and the PageRank sum is about 1.0.

What to say to the teacher
"Indexing is MapReduce: map = tokenize each page, reduce = count words per page. That gives (term, doc, tf). For updates I diff the old and new Iceberg snapshots and re-index only the changed pages. PageRank runs 20 rounds with d = 0.85. Pages with no links share their score with everyone, and we checkpoint to HDFS every 5 rounds, so a crash only repeats a few rounds. The export only publishes if every check passes, and it uses an atomic HDFS rename."

P4 The Help Desk: Node 4 (Search + UI)

Your job in one sentence: fetch the packed results, load them into memory, score pages with BM25 + PageRank, and show them on a web page.

Your files: everything in `search/`: `app.py` (the brain), `static/index.html` (the web page), `pull_export.sh`, `verify_export.py`, `search.service`. You run **no** Spark or HDFS servers.

1. **Before v1 exists** you can already work on the web page (`static/index.html`). Make it look nice; it's your part to shine.
2. **Start the search program** (only once; it restarts by itself after that). First open `search/search.service` and check `User=` and `WorkingDirectory=` match you.

   ```
   sudo cp search/search.service /etc/systemd/system/
   sudo systemctl daemon-reload
   sudo systemctl enable --now search
   ```

3. When P3 says "v1 is ready":

   ```
   bash search/pull_export.sh
   ```

   It downloads, checks, switches `/serving/current` to the new version, and tells the program to reload.
4. Open `http://node4:8000/` in a browser. Search the team's 5 test words and take screenshots (they should say **index v1**).
5. After Phase 1, run `pull_export.sh` again. The same words now show **index v2** and the results change.
6. **Safety demo:** ask for a version that doesn't exist:

   ```
   bash search/pull_export.sh 999
   curl http://localhost:8000/health
   ```

   It fails, and the old version is **still** being served.
7. **Before/after demo:** `bash search/pull_export.sh 1` goes back to v1, and `bash search/pull_export.sh 2` goes forward to v2.

You did it right if...
the web page shows results with **title, doc_id, URL and score**, and `curl "http://localhost:8000/search?q=water"` returns JSON.

Uh-oh: no results for an obvious word?
The two tokenizers aren't twins. Run `bash scripts/sync_textutil.sh`, then `sudo systemctl restart search`.

What to say to the teacher
"Node 4 never talks to Spark or Iceberg. It only reads the checked Parquet export into memory, as a dictionary of term → (docs, tf). For each query I compute BM25 with k1 = 1.2 and b = 0.75, divide by the best BM25, mix in PageRank divided by the best PageRank, and use 0.7 / 0.3 weights. A new version goes live only if it downloads, passes verification and loads. Otherwise we keep the old one."

4. The plan, day by day

Day	P1	P2	P3	P4
1 together	ZeroTier, `cluster.env`, `setup_hosts`, `install_node`, start Node 2 then Node 3, `check_cluster`, `init_tables`, look at the data.			
2 alone	Phase 0 ingest, study the rejects, read `ingest.py`	Practise snapshots, diff and rollback; read the configs	Run index, PageRank, export on Phase 0; read the three jobs	Build the UI; set up the service; read `app.py`
3 together	**Phase 0 end to end:** P1 ingests → P3 processes → P4 pulls → search the 5 test words → screenshots.			
4 together	**Phase 1 + tricks:** changes, the idempotent re-run, v2, time travel, rollback, kill a helper, kill a toy box, the failed-export safety check.			
5 together	**Rehearse.** Write your part of the report. **Swap roles and quiz each other**: the teacher can ask anyone anything!			

5. Show day: the 15-minute demo

1. P2 runs `check_cluster.sh`: 2 DataNodes, 2 workers, repl=2. Opens the HDFS web page.
2. P1 runs Phase 0 ingestion and shows the reject reasons.
3. P3 runs `node3_process.sh` and shows PageRank settling and the Spark history page.
4. P4 runs `pull_export.sh` and does a live search on **v1**.
5. P1 → P3 → P4 repeat everything with **Phase 1**. Same words, new results, **v2**. P1 re-runs Phase 1 to show "nothing changes".
6. P2 shows the snapshot list, the S0 → S1 diff, and explains rollback.
7. P3 + P2 do the fault tolerance demos: kill a Spark helper, then stop a DataNode.
8. P4 shows that a bad export doesn't break search, and switches v2 → v1 → v2.

Show-day checklist (the night before)

All 4 machines charged and on · ZeroTier shows all 4 "online" · `check_cluster.sh` is green · v1 and v2 already exist (just in case) · screenshots saved · snapshot IDs S0 and S1 written on paper.

6. When things break (they will, it's normal)

What you see	What to do
Live datanodes (0)	Re-run `sudo bash scripts/setup_hosts.sh N` on that machine, then `hdfs --daemon stop datanode` and `hdfs --daemon start datanode`.
"Incompatible clusterIDs" in the DataNode log	Someone formatted the NameNode again. On that DataNode: `rm -rf /data/hdfs/datanode/*`, then start the DataNode.
Job stuck on **"Initial job has not accepted any resources"**	No helpers, or they can't call back to the boss. Check `http://node3:8080` shows 2 workers, and that `sudo ufw status` allows the ZeroTier range.
"Catalog 'lake' not found" or Iceberg ClassNotFound	The Iceberg jar is missing on that node: `ls /opt/spark/jars/iceberg*`. Re-run `install_node.sh`.
ModuleNotFoundError: common / textutil	Always use `bash scripts/submit.sh ...` from inside `~/dse`.
Out of memory	Small VMs: lower `spark.executor.memory` to `512m` in `/opt/spark/conf/spark-defaults.conf` and `SPARK_WORKER_MEMORY` to `1g` in `spark-env.sh`, then restart the workers.
Export says **"index is stale"**	Not a bug. It's protecting you. Run index and PageRank first (just run `node3_process.sh`).
Search page says **503 / No index loaded**	**P4**: run `bash search/pull_export.sh`.
A computer was restarted	Node 2: `bash scripts/start_node.sh 2`. Node 3: `bash scripts/start_node.sh 3`. Node 4: the service restarts by itself. Node 1: nothing to do.
Totally stuck	Copy the last 30 lines of the newest file in `~/dse/logs/` and ask for help with that exact error.

7. Last words

Do

- Read **your own files** line by line.
- Explain your part to a teammate until they get it.
- Tell the group before touching shared files.
- Save screenshots of every "it worked!" moment.

Don't

- Format the NameNode again.
- Turn off Node 2 or Node 3 while someone runs a job.
- Edit just one of the two `textutil.py` files.
- Say "the code just does it" in the viva. Know *why*.

You've got this. Four nodes, four friends, one little Google.